

Arogya-SMC Prototype Development Plan (13 Days to March 23)

Technology Stack (Prototype)

Module	Technology	Hosting / Run
Backend API	Node.js + Express	Render (free) or local
Admin Dashboard	Next.js (React) + Leaflet + Recharts	Vercel (free)
Hospital Portal	Next.js (React)	Vercel (free)
ASHA Mobile App	Flutter	Android emulator / physical device
Citizen Mobile App	Flutter	Android emulator / physical device
Database	In-memory / lowdb (JSON file)	Bundled with backend

Module Feature Breakdown

1. Admin Dashboard (Municipal Health Command Centre)

Purpose: Show ward-level heatmap, facility stress, and alerts to the Municipal Health Officer.

Feature	Priority	Description	Acceptance Criteria
Login screen	Must	Simple login form (hardcoded user "mho@smc.gov / password")	After correct credentials, redirect to overview
Overview page	Must	Displays stats cards (total wards, high-risk wards, active alerts, facilities reporting)	Stats come from backend API; show dummy data initially
Recent alerts list	Must	List of 3-5 most recent alerts with severity colour and ward	Data from /api/dashboard/alerts
Trend chart	Must	Line chart of fever/cough cases over last 7 days (using Recharts)	Data from /api/dashboard/trends
Ward heatmap page	Must	Interactive Leaflet map with ward polygons coloured by risk score	Fetch GeoJSON from backend (static for prototype)
Facility status page	Must	Grid of hospitals with bed/ICU/ventilator/oxygen status	Data from /api/public/facilities
Alerts page	Should	Full list of all alerts with filters	Pagination optional
Drill-down ward detail	Could	Click on ward to see detailed indicators	If time permits

2. Hospital Capacity Reporting Portal

Purpose: Allow hospital administrators to submit daily capacity and disease counts.

Feature	Priority	Description	Acceptance Criteria
Login screen	Must	Hospital-specific login (hardcoded users: civil@hospital / password)	Redirect to submission form
Submission form	Must	Form fields: total beds, available beds, ICU beds, available ICU, ventilators, oxygen (yes/no), disease counts (malaria, dengue, etc.)	All fields validated; POST to /api/hospital/capacity
Last submission summary	Should	Show previous submission values for quick update	Fetch from backend using hospital ID
Submission history	Could	List of past submissions	If time permits
Audit logging	Could	Log each submission (backend only)	Optional for prototype

3. ASHA Field Reporting Interface (Mobile App)

Purpose: Enable ASHA workers to submit syndromic reports and receive alerts.

Feature	Priority	Description	Acceptance Criteria
Login / OTP simulation	Must	Simple screen to enter phone number (hardcoded OTP "123456")	After OTP, go to home
Daily report form	Must	Input fields: fever count, cough count, diarrhoea count, maternal risk flag, child risk flag, environmental flag (stagnant water). Geolocation can be simulated.	On submit, send POST to /api/asha/report
Offline storage	Should	Store report in SQLite if no network; background sync when online	Implement using sqflite and connectivity_plus
Alert list	Should	Display push notifications / alerts received from backend	Poll or FCM (simulate with local list)
Sync status indicator	Could	Show pending reports and sync progress	If offline implemented

4. Citizen Health Access & Engagement App (Mobile App)

Purpose: Let citizens find nearby hospital availability and receive advisories.

Feature	Priority	Description	Acceptance Criteria
Ward selection	Must	Dropdown to select ward (or simulate GPS with manual entry)	On selection, fetch facilities
Facility list	Must	List of hospitals in that ward with bed/ICU/oxygen availability	Data from <code>/api/public/facilities?ward=...</code>
Facility detail	Should	Tap on hospital to see more info (contact, address)	
Advisories feed	Must	List of public health advisories (from backend)	Data from <code>/api/public/advisories</code>
Push notifications	Could	Simulate receiving an alert (e.g., via local notification)	If time permits

5. Backend API & Services

Purpose: Provide REST endpoints for all frontend modules, store data, and generate simple alerts.

Endpoint	Method	Description	Data Format	Auth
<code>/api/asha/report</code>	POST	Receive ASHA report	<code>{ workerId, ward, date, fever, cough, diarrhea, flags, location }</code>	JWT (simulate with token)
<code>/api/hospital/capacity</code>	POST	Receive hospital capacity	<code>{ hospitalId, date, beds, icu, ventilators, oxygen, diseaseCounts }</code>	JWT
<code>/api/public/facilities</code>	GET	List facilities with latest capacity	Query param <code>ward</code> optional	None
<code>/api/public/advisories</code>	GET	List current advisories	–	None
<code>/api/dashboard/summary</code>	GET	Ward-level summary stats	–	JWT (admin)
<code>/api/dashboard/trends</code>	GET	Time-series data for charts	–	JWT (admin)

Endpoint	Method	Description	Data Format	Auth
/api/dashboard/alerts	GET	List active alerts	–	JWT (admin)
/api/wards/geojson	GET	GeoJSON of ward boundaries (static)	–	None (or admin)

Additional Services:

- **Simple alert engine:** On each new report, check if fever count in a ward exceeds threshold (e.g., 30). If so, create an alert in the alerts list.
- **Data storage:** Use lowdb (JSON file) for persistence between restarts (optional). For prototype, in-memory is acceptable if we seed data.

Authentication:

- For ASHA and hospital, we'll use JWT tokens obtained from a simple [/api/login](#) endpoint (hardcoded user DB). For demo, we can skip real JWT and use a fixed token or session cookie.

Development Roadmap (13 Days)

Day	Focus	Tasks
1	Setup	Create GitHub repo, initialize backend (Node/Express), Next.js projects for admin and hospital, Flutter projects for ASHA and citizen. Define API contract.
2	Core Backend	Implement basic Express server with in-memory storage. Create endpoints for ASHA report, hospital capacity, public facilities, and dashboard summary. Seed with sample data.
3	Admin Dashboard (basic)	Build login page, overview page with static cards and chart (later connect to API). Start Leaflet map with static GeoJSON.
4	Hospital Portal	Build login and submission form. Connect to backend (POST). Test with Postman.
5	ASHA App (basic)	Build login screen and report form. Implement API call to submit report (online only).
6	Citizen App (basic)	Build ward selection screen, fetch facilities from API and display list.
7	Integration	Connect all frontends to actual backend. Test end-to-end flows: ASHA submits → admin sees alert. Hospital updates capacity → citizen sees change.
8	Alerts & Polish	Implement simple alert engine in backend. Add alerts list to admin dashboard. Polish UI (loading states, error handling).

Day	Focus	Tasks
9	Offline (ASHA)	If time, implement SQLite storage and background sync in ASHA app.
10	Deployment	Deploy backend to Render, frontends to Vercel. Prepare environment variables.
11	Testing	Full system test on all modules. Fix critical bugs.
12	Demo Rehearsal	Run through the demo script, record video, ensure smooth flow.
13	Final Submission	Package everything, upload video, double-check declaration.

Team Task Allocation (5 Members)

- **Member A (Backend):** Node.js + Express, database, API design, deployment.
- **Member B (Admin Dashboard):** Next.js, Leaflet, Recharts, API integration.
- **Member C (Hospital Portal):** Next.js forms, API integration.
- **Member D (ASHA Mobile):** Flutter, form handling, offline storage (if time).
- **Member E (Citizen Mobile):** Flutter, location simulation, list views.

All members should collaborate on API contract and help with testing.

Risk Mitigation

- **Backend not ready:** Frontends can use mock data initially and switch to real API later.
- **Mobile development slow:** Focus on core screens first; use simulators.
- **Deployment issues:** Have local demo as fallback (record video locally).
- **Offline complexity:** If offline storage takes too long, skip and simulate online-only for demo.